Chapter 1 Basics concepts of hydrological models: routing

Olivier Bonte

2026-07-01

To get more insight into the routing concepts explained in the theory, a computational guide is given here

1 Reservoir cascade models

1.1 Cascade of linear reservoirs

1.1.1 First order linear reservoir with constant input

A linear reservoir with storage S [m^3] and a constant input I [m^3/s] is characterised by

$$S = KQ \tag{1}$$

with K [s] the residence time and Q [m^3/s] the outflow. The mass balance equation is given by

$$\frac{dS}{dt} = K\frac{dQ}{dt} = I - Q \tag{2}$$

As this is a linear ordinary differential equation, it can be solved analytically using Laplace transforms (see your course in the 2nd bachelor year on Differential Equations). Remember that although the definition of a Laplace transform might seem complex

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)e^{-st}dt,$$

in practice you only need to apply a limited set of transforms to solve linear ODEs, which are summarised in the table below.

$f(t)$	$F(s) = \mathcal{L}\{f(t)\}$
$H(t)$	$\frac{1}{s}$
t^n	$\frac{n!}{s^{n+1}}$
e^{at}	$\frac{1}{s-a}$
$f'(t)$	$sF(s) - f(0)$

where $H(t)$ is the Heaviside function, which is defined as $H(t) = 0$ for $t < 0$ and $H(t) = 1$ for $t \geq 0$. n is a positive integer and a is a real number. Applying the Laplace transform to equation Equation 2 gives with $Q(t = 0) = Q_0 = S_0/K$:

$$K(sQ(s) - Q_0) = \frac{I}{s} - Q(s)$$

Isolating $Q(s)$ gives:

$$Q(s) = \frac{I}{s(1 + Ks)} + \frac{KQ_0}{1 + Ks}$$

We now need to apply the inverse Laplace transform to get $Q(t)$. Trivially, $\mathcal{L}^{-1}(Q(s)) = Q(t)$. For the term containing Q_0:

$$\mathcal{L}^{-1}\left(\frac{KQ_0}{1 + Ks}\right) = \frac{K}{K}Q_0\mathcal{L}^{-1}\left(\frac{1}{1/K + s}\right) = Q_0 e^{-t/K} \tag{3}$$

The term with I is a bit more convolved, but can be solved using partial fraction decomposition:

$$\frac{I}{s(1 + Ks)} = \frac{A}{s} + \frac{B}{1 + Ks}$$

To solve for A, multiply both sides by s and set $s = 0$:

$$\frac{I}{1 + Ks}\Big|_{s=0} = A \iff A = I$$

To solve for B, multiply both sides by $(1 + Ks)$ and set $s = -1/K$:

$$\frac{I}{s}\Big|_{s=-1/K} = B \iff B = -IK$$

We now need to apply the inverse Laplace transform to get $Q(t)$. Trivially, $\mathcal{L}^{-1}(Q(s)) = Q(t)$. For the term containing Q_0:

$$\mathcal{L}^{-1}\left(\frac{KQ_0}{1+Ks}\right) = \frac{K}{K}Q_0\mathcal{L}^{-1}\left(\frac{1}{1/K+s}\right) = Q_0 e^{-t/K} \tag{4}$$

The term with I is a bit more convolved, but can be solved using partial fraction decomposition:

$$\frac{I}{s(1+Ks)} = \frac{A}{s} + \frac{B}{1+Ks}$$

To solve for A, multiply both sides by s and set $s = 0$:

$$\frac{I}{1+Ks}\Big|_{s=0} = A \iff A = I$$

To solve for B, multiply both sides by $(1+Ks)$ and set $s = -1/K$:

$$\frac{I}{s}\Big|_{s=-1/K} = B \iff B = -IK$$

Now we can apply the inverse laplace transform on the partial fraction decomposition:

$$\mathcal{L}^{-1}\left(\frac{I}{s} - \frac{IK}{1+Ks}\right) = \mathcal{L}^{-1}\left(\frac{I}{s} - \frac{I}{s+1/K}\right) = I - Ie^{-t/K} = I(1-e^{-t/K}) \tag{5}$$

Combining Equation 4 and Equation 5 gives the final solution for $Q(t)$:

$$Q(t) = I(1-e^{-t/K}) + Q_0 e^{-t/K} \tag{6}$$

E.g. for $Q_0 = 0.1$ m^3/h and $K = 7$ h, we can plot the outflow $Q(t)$ for different constant inflows I. In the examples that follow, we'll use m^3/h and h for the units instead of the standard SI-units for convenience. Define the analytical solutions as a function:

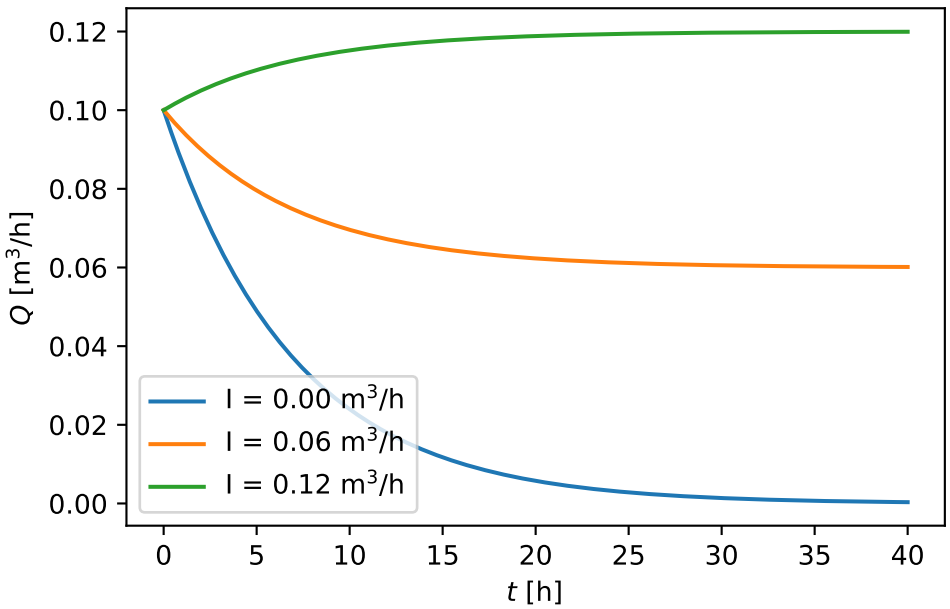
```
import matplotlib.pyplot as plt
import numpy as np


def Q_analytical_linear(t, I, K, Q0):
    return I * (1 - np.exp(-t / K)) + Q0 * np.exp(-t / K)
```

define the parameters and the three different inflows:

```
Q_0_num = 0.1 # m^3/h
K_num = 7 # h
t_array = np.linspace(0, 40, 200)  # h
I_array = [0.0, 0.06, 0.12]  # m^3/h
```

to then plot the results:

```
fig, ax = plt.subplots()
colors = plt.rcParams["axes.prop_cycle"].by_key()["color"]
for i, I_ in enumerate(I_array):
    Q_ = Q_analytical_linear(t_array, I_, K_num, Q_0_num)
    ax.plot(t_array, Q_, color=colors[i], label=rf"I = {I_:.2f} m$^3$/h")
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```



Additionally, note that we can automate the algebraic routines above using SymPy. First define all symbols:

```
import sympy as sp

t, K, I, Q0 = sp.symbols("t K I Q0", positive=True)
s = sp.symbols("s")
```

```
q, Q = sp.symbols("q, Q", cls=sp.Function)
```

and then define the ODE in the form $K\frac{dQ}{dt} - (I - Q) = 0$:

```
ode_linear = K * sp.diff(Q(t), t) - (I - Q(t))
ode_linear
```

$$-I + K\frac{d}{dt}Q(t) + Q(t)$$

Now take the Laplace transform of the ODE defining $q(s) = \mathcal{L}\{Q(t)\}$:

```
ode_linear_laplace = sp.laplace_transform(ode_linear, t, s, noconds=True)
ode_linear_laplace = sp.laplace_correspondence(ode_linear_laplace, {Q: q})
ode_linear_laplace
```

$$-\frac{I}{s} + K\left(sq(s) - Q(0)\right) + q(s)$$

We now insert the initial condition $Q(0) = Q_0$:

```
ode_linear_laplace_ic = sp.laplace_initial_conds(ode_linear_laplace, t, {Q:
↪  [Q0]})
ode_linear_laplace_ic
```

$$-\frac{I}{s} + K\left(-Q_0 + sq(s)\right) + q(s)$$

In the equation above, the only unknown is $q(s)$, which can be solved for:

```
q_s_solved = sp.solve(ode_linear_laplace_ic, q(s))[0]
q_s_solved
```

$$\frac{I + KQ_0 s}{s\left(Ks + 1\right)}$$

Now apply the inverse Laplace transform to get $Q(t)$:

```
Q_solved = sp.simplify(sp.inverse_laplace_transform(q_s_solved, s, t))
Q_solved
```

$$\left(Ie^{\frac{t}{K}} - I + Q_0\right)e^{-\frac{t}{K}}$$

Which is identical to the analytical solution Equation 6 derived above.

1.1.2 Second order linear reservoir with constant input

We now get two reservoirs in series:

$$K^{(1)}\frac{dQ^{(1)}}{dt} = I - Q^{(1)}$$
$$K^{(2)}\frac{dQ^{(2)}}{dt} = Q^{(1)} - Q^{(2)}$$

The same SymPy approach can be extended to solve this coupled system. To simplify the solution, we'll assume:

- The second reservoir is initially empty, i.e. $Q^{(2)}(0) = 0$.
- The first reservoir has an initial outflow $Q^{(1)}(0) = Q_0$ as it is assumed non-empty.
- The first and second reservoirs have the same residence time, i.e. $K^{(1)} = K^{(2)} = K$.

We define two unknown functions and their Laplace transforms:

```
q1, q2, Q1, Q2 = sp.symbols("q1 q2 Q1 Q2", cls=sp.Function)
```

Write both ODEs in the form $(\dots) = 0$:

```
ode1 = K * sp.diff(Q1(t), t) - (I - Q1(t))
ode2 = K * sp.diff(Q2(t), t) - (Q1(t) - Q2(t))
display(ode1, ode2)
```

$$-I + K\frac{d}{dt}Q_1(t) + Q_1(t)$$

$$K\frac{d}{dt}Q_2(t) - Q_1(t) + Q_2(t)$$

Take the Laplace transform using $q_1(s) = \mathcal{L}\{Q_1(t)\}$ and $q_2(s) = \mathcal{L}\{Q_2(t)\}$:

```
ode1_laplace = sp.laplace_transform(ode1, t, s, noconds=True)
ode1_laplace = sp.laplace_correspondence(ode1_laplace, {Q1: q1})

ode2_laplace = sp.laplace_transform(ode2, t, s, noconds=True)
ode2_laplace = sp.laplace_correspondence(ode2_laplace, {Q1: q1, Q2: q2})
```

Insert the initial conditions:

```
ode1_laplace_ic = sp.laplace_initial_conds(ode1_laplace, t, {Q1: [Q0], Q2:
↪  [0]})
ode2_laplace_ic = sp.laplace_initial_conds(ode2_laplace, t, {Q1: [Q0], Q2:
↪  [0]})
display(ode1_laplace_ic, ode2_laplace_ic)
```

$$-\frac{I}{s} + K\left(-Q_0 + sq_1(s)\right) + q_1(s)$$

$$Ksq_2(s) - q_1(s) + q_2(s)$$

Solve the algebraic system for $q_1(s)$ and $q_2(s)$:

```
q_s_solved = sp.solve([ode1_laplace_ic, ode2_laplace_ic], [q1(s), q2(s)],
↪  dict=True)[0]
q2_s = q_s_solved[q2(s)]
q2_s
```

$$\frac{I + KQ_0 s}{K^2 s^3 + 2Ks^2 + s}$$

To get a solution closer to the notation of the theory, we'll first break up into partial fractions:

```
q2_s_partial = sp.apart(q2_s, s)
q2_s_partial
```

$$-\frac{IK}{Ks+1} + \frac{I}{s} - \frac{K\left(I - Q_0\right)}{\left(Ks+1\right)^2}$$

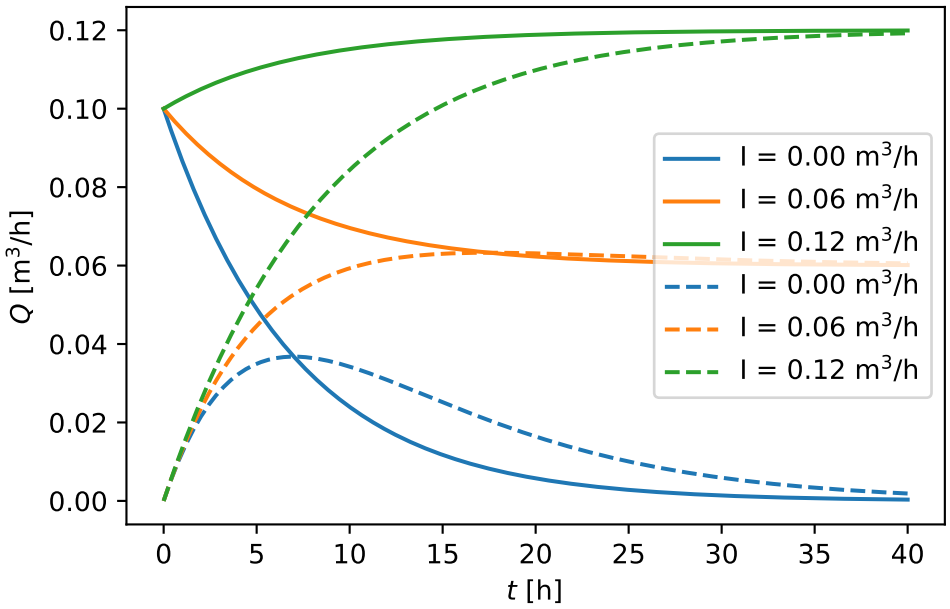
Finally, apply the inverse Laplace transform to obtain $Q_2(t)$.

```
Q2_solved = sp.inverse_laplace_transform(q2_s_partial, s, t)
Q2_solved
```

$$I - Ie^{-\frac{t}{K}} - \frac{t\left(I - Q_0\right)e^{-\frac{t}{K}}}{K}$$

Also $Q_2(t)$ can now be visualised for different constant inflows I and the same parameters as before:

```
for i, I_ in enumerate(I_array):
    Q2_func = sp.lambdify(t, Q2_solved.subs({I: I_, K: K_num, Q0: Q_0_num}),
 ↪  "numpy")
    Q2_ = Q2_func(t_array)
    ax.plot(
        t_array, Q2_, label=rf"I = {I_:.2f} m$^3$/h", color=colors[i],
 ↪  linestyle="--"
    )
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
fig
```



In this figure, full lines are the output of the first reservoir and dashed lines are the output of the second reservoir. Note that the second reservoir has a delayed response compared to the first reservoir.

Given that the second reservoir is initially empty and receives no external input I, Q_2 is entirely defined by Q_1 and $K^{(1)} = K$. So if you solve for Q_2 (in the s-domain $q_2(s)$) with an assumed known $Q_1(t)$ function (in the s-domain $q_1(s)$), you get:

```
q2_s_bis = sp.solve(ode2_laplace_ic, q2(s))[0]
q2_s_bis
```

$$\frac{q_1(s)}{Ks+1}$$

This concept can be generalised in what is called a **transfer function**, which is defined as the ratio of the Laplace transforms of the output and input of a system. In this case, the transfer function $H(s)$ is defined as:

$$H(s) = \frac{q_2(s)}{q_1(s)} = \frac{1}{1+Ks}$$